

In this homework, we handle with the implementation of AVL tree. Because we want time complexity $O(\log n)$. We implemented the insert, delete and search method recursively. Also HeightDiff, GetBalance, Findmin, RotateRight, RotateLeft methods are used as helper methods. On the other hand, preorder, postorder, inorder methods were implemented to print AVL tree.

Insertion Method:

.In this method, implementation of this method is almost the same with the binary tree insertion method, but it is a little bit different because after each node insertion, we try to balance the tree using HeightDiff method, RotateRight, RotateLeft methods.

.Firstly, we look this node was added or not, if it is, it updates the values, if not add the method.

.According to the HeightDiff method result, we decide which rotations will be done. Rotate Right-Right, Rotate Left-Right, Rotate Right-Left, Rotate Left-Left.

.Also for each node, its height is updated.

Remove Method:

.In this method, first we check to determine whether node is valid or not. If not, print error message, else deletion is made.

.We use findmin, getbalance methods as helper methods.

.When we find the node, we look its children. According to its children, necessary shift operation is made.

Search Method:

.In this method, we find the node with the name of stock. It traverses the tree as recursive.

WriteToFileMethod

.In this method, we generate random values for volume, marketcap, name and price. Names have three or four letters. Randomly all of them is generated between 200000-800000. All these values are written to input.txt file.

PerformAnalysisMethod

.In this method, txt file is read and according to the type of commands, it creates the AVL tree. Also it calculates the time for the command type. (For example, if it is add, it will

calculate the time of insertion one node. Program keeps this value as Map. Then graphic is drawn.

AverageTimeCalculatorMethod

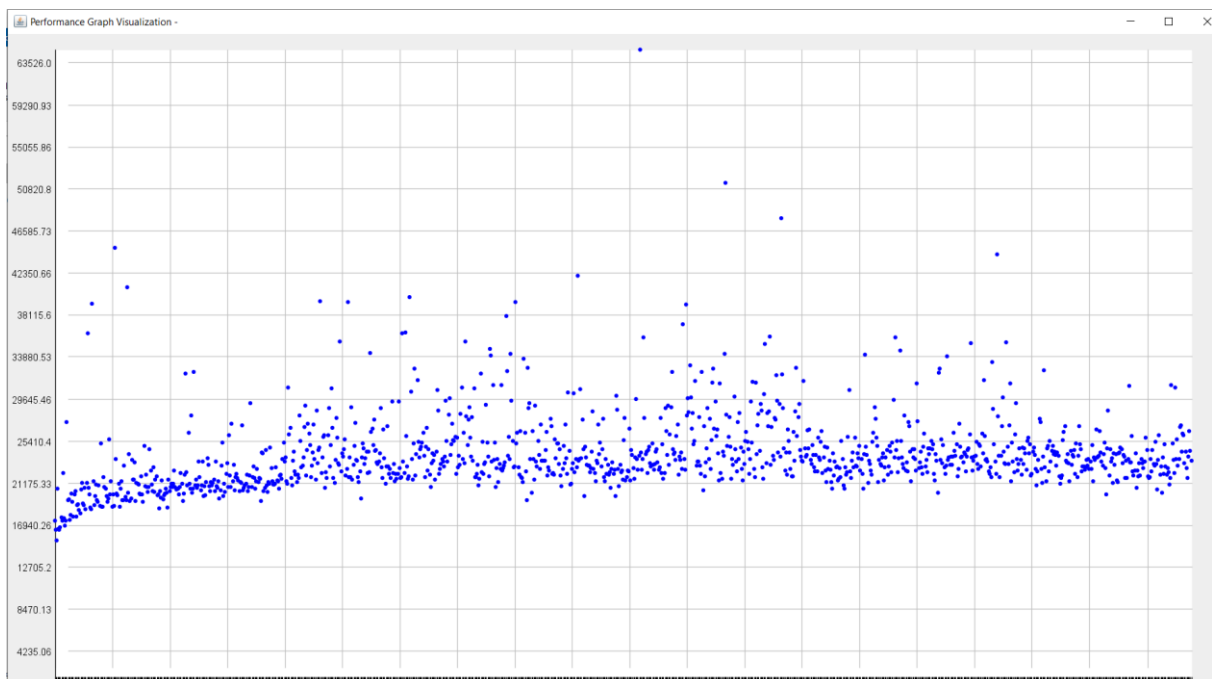
.In this method, we consider about key entries of map. According to the each key, we have an output which is the time of that node operation (endtime-starttime). So, we sum the each value and we divide the total sum to map size. So we calculate the average time of search, add, deletion.

GenerateRandomStringMethod

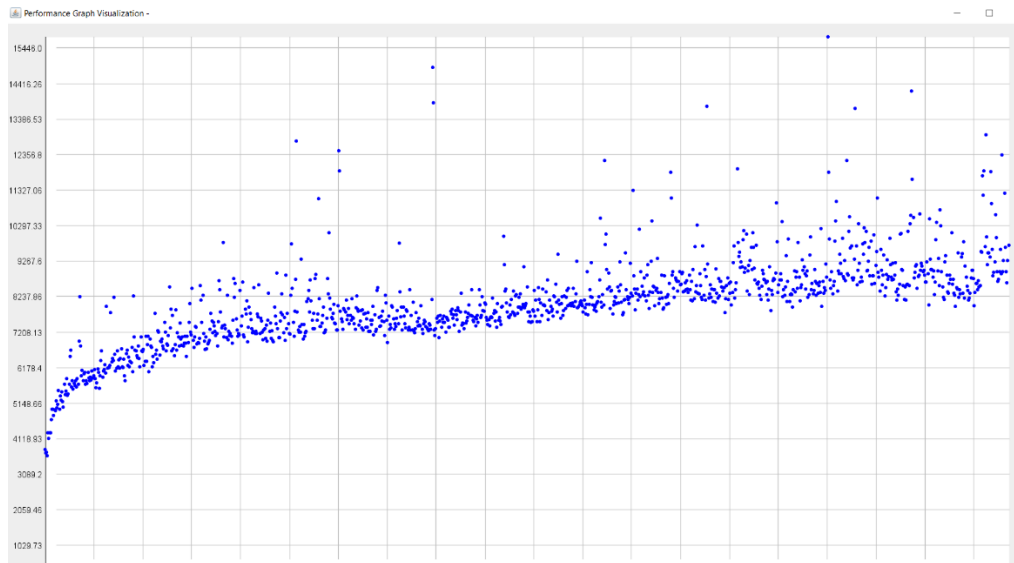
.This method is used in writeToFileMethod. it generates random name.

PerfromAnalaysisOfOperations

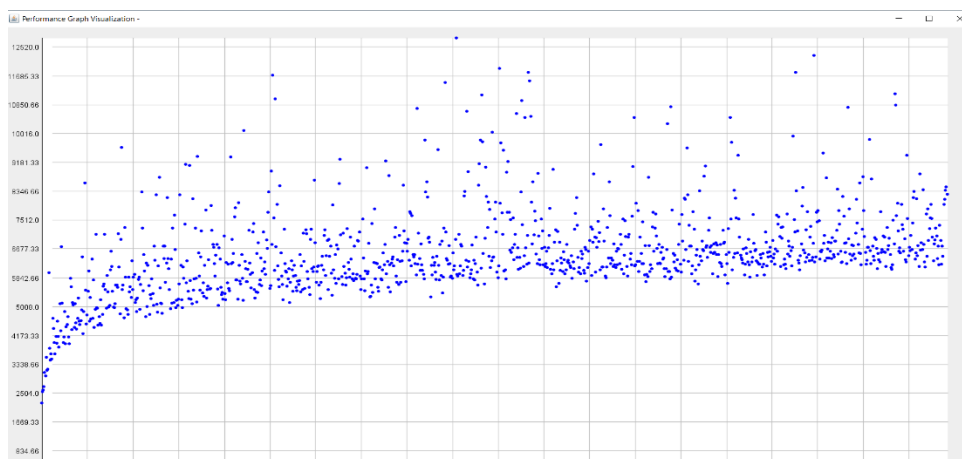
Insertion Method: It is expected to see $O(\log n)$ graph. Also number of times is divided to 15 to see more valid graph.



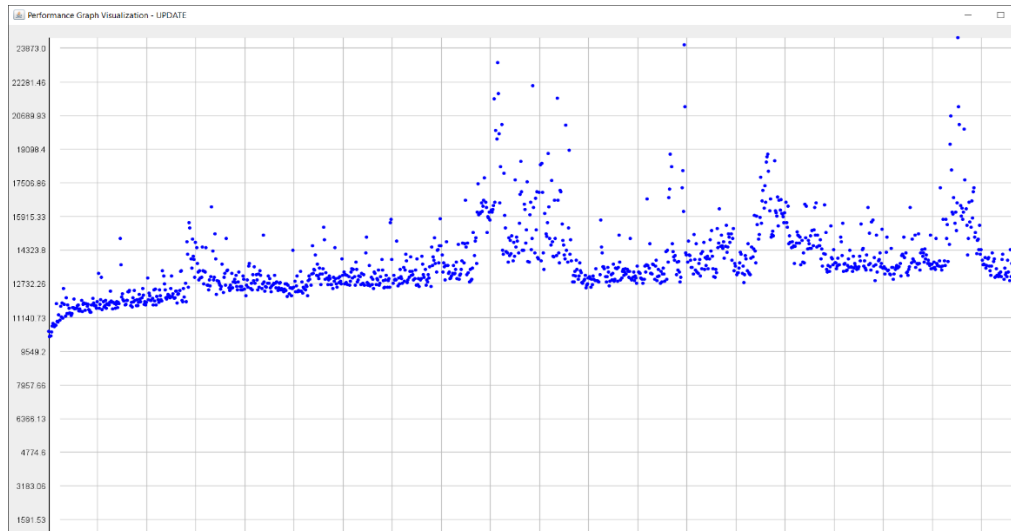
Remove Method: It is expected to see $O(\log n)$ graph. Also number of times is divided to 15 to see more valid graph.



Search method: It is expected to see $o(\log n)$ graph. Also number of times is divided to 15 to see more valid graph.



Update Method: It is expected to see $O(\log n)$ graph. Also number of times is divided to 15 to see more valid graph.



What was the hard things in homeworks:

1) observing the $O(\log n)$ graph was hard. We tried to keep times in an arraylist but the performance was not enough. So i used the Map to keep times. Also makefile was different from previous. -Xint is used.

2) Also we need huge datas in txt file. I writed an WriteToFileMethod so i can have lots of command in txt file